

Introduction to Python Programming

Chapter 11: Functions - Introduction

Introduction

Functions are one of the most powerful tools in programming. They allow you to write code once and reuse it many times. Functions help organize code, make it easier to read, reduce repetition, and make debugging simpler. This chapter introduces the fundamentals of creating and using functions.

1. What is a Function?

A **function** is a reusable block of code that performs a specific task. Think of it as a mini-program within your program.

Why use functions?

- **Avoid repetition:** Write code once, use it many times
- **Organization:** Break complex programs into manageable pieces
- **Easier debugging:** Fix problems in one place
- **Readability:** Functions with good names make code self-documenting

Built-in Functions You Already Know

You've been using functions all along:

```
print("Hello")      # print is a function
len([1, 2, 3])      # len is a function
int("42")           # int is a function
input("Name: ")     # input is a function
```

Now you'll learn to create your own!

2. Creating Your First Function

Basic Function Structure

```
def function_name():
    # code block
    # this code runs when the function is called
```

Key parts:

- `def` keyword starts the definition
- Function name (follow same rules as variables)
- Parentheses `()` (empty for now)
- Colon `:`
- Indented code block

Simple Function Example

```
def greet():  
    print("Hello!")  
    print("Welcome to Python functions!")  
    print("Let's learn together!")  
  
# Call the function to run it  
greet()
```

Output:

```
Hello!  
Welcome to Python functions!  
Let's learn together!
```

Calling Functions Multiple Times

```
def greet():  
    print("Hello!")  
  
greet() # First call  
greet() # Second call  
greet() # Third call
```

Output:

```
Hello!  
Hello!  
Hello!
```

The function code runs each time you call it.

3. CRITICAL: Function Definition Order

Functions must be defined BEFORE they are called.

This Works ✓

```
def greet():  
    print("Hello!")  
  
greet() # Function defined above, this works!
```

This Breaks ✗

```
greet() # ERROR! Function not defined yet  
  
def greet():  
    print("Hello!")
```

Why This Matters

Python reads your code from top to bottom. When it encounters `greet()`, it needs to know what `greet` is.

Think of it this way:

1. Python reads the `def greet():` statement → "Okay, I know what greet is now"
2. Python stores the function definition in memory
3. Later, when Python sees `greet()` → "I remember greet! I'll run that code"

Important: Defining a function doesn't run it! The code inside only runs when you call the function.

```
def greet():  
    print("This doesn't print yet!")  
  
# Code above runs, but nothing prints  
  
greet() # NOW it prints!
```

4. Functions with Parameters

Parameters let you pass information into a function.

Parameter vs Argument - Critical Distinction

Parameter: Variable in the function definition (the placeholder)

Argument: Actual value you pass when calling (the real data)

```
def greet(name):      # 'name' is a PARAMETER
    print(f"Hello, {name}!")

greet("Alice")        # "Alice" is an ARGUMENT
```

Think of it this way:

- **Parameter** = the container/slot
- **Argument** = what you put in the container

Simple Parameter Example

```
def greet(name):
    print(f"Hello, {name}!")
    print(f"Nice to meet you, {name}!")

greet("Alice")
greet("Bob")
greet("Charlie")
```

Output:

```
Hello, Alice!
Nice to meet you, Alice!
Hello, Bob!
Nice to meet you, Bob!
Hello, Charlie!
Nice to meet you, Charlie!
```

Multiple Parameters

```
def introduce(name, age):
    print(f"My name is {name}")
    print(f"I am {age} years old")

introduce("Alice", 25)
introduce("Bob", 30)
```

Output:

```
My name is Alice
I am 25 years old
My name is Bob
I am 30 years old
```

Important: Arguments are passed in order! First argument goes to first parameter, second to second, etc.

5. Real-World Example: Range Checking

Let's create a function that prints numbers from 4 to some ending number. But what if the user passes a number less than 4? We need to handle that!

Version 1: Count Up Only

```
def count_from_four(end_number):
    if end_number < 4:
        print("Error: Number must be 4 or greater")
    elif end_number == 4:
        print("4")
    else:
        for i in range(4, end_number + 1):
            print(i)

count_from_four(7)    # Prints: 4 5 6 7
count_from_four(2)    # Prints: Error message
count_from_four(4)    # Prints: 4
```

Why the checks?

- If `end_number < 4`: Can't count from 4 to 2 (going up)
- If `end_number == 4`: Just print 4
- Otherwise: Print from 4 to `end_number`

Version 2: Count Backwards If Needed

```
def count_from_four(end_number):
    if end_number == 4:
        print("4")
    elif end_number > 4:
        # Count up
        for i in range(4, end_number + 1):
            print(i)
    else:
        # Count down
        for i in range(4, end_number - 1, -1):
            print(i)

count_from_four(7)    # Counts up: 4 5 6 7
count_from_four(1)    # Counts down: 4 3 2 1
count_from_four(4)    # Just: 4
```

Key lesson: When writing functions, think about all possible inputs. What if the number is too small? Too large? Exactly the boundary? Handle edge cases!

6. Return Values

Functions can send values back to where they were called using `return`.

Basic Return

```
def add_numbers(a, b):  
    result = a + b  
    return result  
  
total = add_numbers(5, 3)  
print(total)  # 8
```

What happens:

1. Function calculates `5 + 3 = 8`
2. `return` sends 8 back
3. The value 8 replaces the function call
4. `total = 8`

Return vs Print

These are different!

```
def add_with_print(a, b):  
    result = a + b  
    print(result)  
  
def add_with_return(a, b):  
    result = a + b  
    return result  
  
# With print  
add_with_print(5, 3)      # Prints 8  
x = add_with_print(5, 3)  # Prints 8, but x is None!  
print(x)                  # None  
  
# With return  
add_with_return(5, 3)     # Returns 8 (nothing prints)  
y = add_with_return(5, 3) # y = 8  
print(y)                  # 8
```

Key difference:

- **Print:** Shows something on screen, returns nothing
- **Return:** Sends a value back, shows nothing

Using Return Values

```
def multiply(a, b):  
    return a * b  
  
# Use the return value in calculations  
result = multiply(4, 5) + 10  
print(result)  # 30  
  
# Use in conditions  
if multiply(3, 4) > 10:  
    print("Greater than 10")  
  
# Pass to another function  
print(multiply(6, 7))
```

7. Functions That Return True/False

Functions can return boolean values for decision-making.

Example: Is Number Even?

Long version (clear for beginners):

```
def is_even(number):  
    remainder = number % 2  
    if remainder == 0:  
        return True  
    else:  
        return False  
  
print(is_even(4))  # True  
print(is_even(7))  # False  
  
if is_even(10):  
    print("10 is even")
```

How it works:

1. Calculate `number % 2` (remainder when divided by 2)
2. If remainder is 0, number is even → return True
3. Otherwise, number is odd → return False

Compact Version (More Pythonic)

Here's where it gets interesting - we can return the condition directly:

```
def is_even(number):  
    return number % 2 == 0
```

Wait, what? Let's break this down carefully:

```
# The expression 'number % 2 == 0' evaluates to True or False  
# If number is 4:  
4 % 2 == 0 # 0 == 0, which is True  
# So: return True  
  
# If number is 7:  
7 % 2 == 0 # 1 == 0, which is False  
# So: return False
```

This works because:

- `number % 2 == 0` is a boolean expression
- It already evaluates to True or False
- We can return that result directly!

Comparing both versions:

```
# Long version - uses if/else  
def is_even_long(number):  
    if number % 2 == 0:  
        return True  
    else:  
        return False  
  
# Short version - returns the condition  
def is_even_short(number):  
    return number % 2 == 0  
  
# Both work exactly the same!  
print(is_even_long(8)) # True  
print(is_even_short(8)) # True
```


Why use the compact version?

- Less code to write
- Easier to read (once you understand it)
- More "Pythonic" (Python's style)

Both are legitimate! Use the long version if it's clearer for you, use the short version when you're comfortable with it.

Another Example: Is Odd?

```
def is_odd(number):  
    return number % 2 == 1  
  
# OR, since odd is "not even":  
def is_odd(number):  
    return number % 2 != 0
```

8. Conditional Returns

You can return different values based on conditions.

Example: Grade Evaluator

```
def get_grade(score):  
    if score >= 90:  
        return "A"  
    elif score >= 80:  
        return "B"  
    elif score >= 70:  
        return "C"  
    elif score >= 60:  
        return "D"  
    else:  
        return "F"  
  
grade = get_grade(85)  
print(f"Your grade: {grade}") # Your grade: B
```

Important: As soon as a `return` runs, the function stops! No code after a `return` in that path executes.

```
def example(x):
    if x > 0:
        return "Positive"
        print("This never prints") # Unreachable!
    return "Not positive"

print(example(5))    # "Positive"
print(example(-3))  # "Not positive"
```

Early Returns for Validation

```
def divide(a, b):
    if b == 0:
        return "Error: Cannot divide by zero"
    return a / b

print(divide(10, 2)) # 5.0
print(divide(10, 0)) # Error: Cannot divide by zero
```

This pattern checks for problems first, returns error messages, then handles the normal case.

9. Practical Examples

Example 1: Temperature Converter

```
def celsius_to_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit

temp_c = 25
temp_f = celsius_to_fahrenheit(temp_c)
print(f"{temp_c}°C = {temp_f}°F")
```

Example 2: Find Maximum

```
def find_max(a, b, c):  
    if a >= b and a >= c:  
        return a  
    elif b >= a and b >= c:  
        return b  
    else:  
        return c  
  
print(find_max(5, 12, 8)) # 12
```

Example 3: Validate Age

```
def is_valid_age(age):  
    if age < 0:  
        return False  
    if age > 120:  
        return False  
    return True  
  
age = int(input("Enter age: "))  
if is_valid_age(age):  
    print("Valid age")  
else:  
    print("Invalid age")
```

Example 4: Calculate Factorial

```
def factorial(n):  
    if n < 0:  
        return "Error: Cannot calculate factorial of negative number"  
    if n == 0 or n == 1:  
        return 1  
  
    result = 1  
    for i in range(2, n + 1):  
        result = result * i  
    return result  
  
print(factorial(5)) # 120  
print(factorial(-2)) # Error message
```

Example 5: Check Prime Number

```
def is_prime(number):
    if number < 2:
        return False

    for i in range(2, number):
        if number % i == 0:
            return False # Found a divisor, not prime

    return True # No divisors found, is prime

print(is_prime(7))    # True
print(is_prime(8))    # False
print(is_prime(13))   # True
```

10. Common Function Patterns

Pattern 1: Transform and Return

```
def double(number):
    return number * 2

def square(number):
    return number ** 2
```

Pattern 2: Validate and Return Boolean

```
def is_positive(number):
    return number > 0

def is_in_range(number, low, high):
    return low <= number <= high
```

Pattern 3: Calculate and Return

```
def area_of_circle(radius):
    pi = 3.14159
    return pi * radius ** 2

def hypotenuse(a, b):
    return (a**2 + b**2) ** 0.5
```

Pattern 4: Check Conditions, Return Result

```
def can_vote(age):  
    if age >= 18:  
        return True  
    return False  
  
# Or more simply:  
def can_vote(age):  
    return age >= 18
```

Key Takeaways

- ✓ **Functions organize code** into reusable blocks
- ✓ **Define before use** - functions must be defined before being called
- ✓ **Defining doesn't run** - function code only runs when called
- ✓ **Parameters vs Arguments:**
 - Parameters: placeholders in definition
 - Arguments: actual values passed when calling
- ✓ **Return sends values back** - different from printing
- ✓ **Return stops execution** - code after return doesn't run
- ✓ **Boolean returns** - can return True/False directly from conditions
- ✓ **Compact boolean returns:** `return condition` instead of if/else
- ✓ **Handle edge cases** - think about all possible inputs
- ✓ **Early returns** - check for problems first, handle normal case last

Reflection Questions

1. What's the difference between defining and calling a function?
2. Why must functions be defined before they're called?
3. What's the difference between a parameter and an argument?
4. What's the difference between print and return?
5. What happens when a return statement executes?
6. How can you return True/False without using if/else?
7. Why check for edge cases in functions?
8. Can a function have multiple return statements?

Practice Exercises

1. **Simple Greeting:** Create a function that prints a greeting message. Call it 3 times.
2. **Add Two Numbers:** Create a function that takes two numbers and returns their sum.
3. **Is Negative:** Create a function that returns True if a number is negative, False otherwise.
4. **Count to N:** Create a function that prints numbers from 1 to N (parameter).
5. **Calculate Average:** Create a function that takes three numbers and returns their average.
6. **Password Validator:** Create a function that checks if a password is at least 8 characters. Return True/False.
7. **Find Minimum:** Create a function that takes three numbers and returns the smallest.
8. **Grade Converter:** Create a function that takes a numeric score and returns a letter grade.
9. **Is Palindrome:** Create a function that checks if a number is the same forwards and backwards (like 121).
10. **Calculate Tax:** Create a function that takes a price and tax rate, returns total with tax.